Robust Humanoid Foundation Model via Task-Aware Failure Detection and Recovery

Anonymous Authors

***Abstract*— Foundation models show promise in generalizability, but they do not maintain robustness outside of the distribution of their training demonstrations. As a result, these policies need mechanisms for failure prediction and correction at runtime. Additionally, the robotic systems that we aim to deploy these policies on, such as humanoids, are vastly complex. This makes generic task-level failure recognition challenging. We propose an agentic approach to humanoid foundation model failure detection and recovery that provides a vision-language orchestrator model access to several executable robot control behaviors referred to as recovering tools. This ensemble approach is capable of differentiating between pre-task and mid-task failures via systematic policy perturbations and zero-shot sim2real transfer of a learned pre-task failure detector. The recovery tools further decompose corrections into locomotion- and manipulation-level behaviors. To further improve the capabilities of these recovery tools at run-time, we train a residual network on teleoperation demonstrations and apply learned position and orientation offsets for grasps to align policy and tool actions. We deploy this framework on the Unitree G1 humanoid and observe a 50% improvement in task success rate and enhanced operational robustness.**

I. Introduction

Humanoid foundation models have demonstrated the ability to perform complex tasks ranging from bi-manual manipulation [1]–[3] to whole-body loco-manipulation [4]. Common training recipes for these models include Vision-Language-Model (VLM) backbone pre-training, action expert post-training, and task-specific fine-tuning. This pipeline reduces the diversity of the training data as these stages advance, with fine-tuning typically being done for a singular task on a singular embodiment in a singular environment. This process leads to optimal task performance, but it necessarily reduces the operating regime of the models themselves.

Given the nebulous operating conditions of these monolithic models, robust online deployment requires both failure detection and failure recovery. Foundation models alone can not be trusted to repeat tasks with sufficient levels of reliability required to accept them into the real world as operators alongside humans. Failure detection concerns identifying when the model in question encounters out-of-distribution input data. Observations for foundation models are often multi-modal in nature, and humanoids are high-dimensional, non-stationary systems. This makes it difficult to identify what internal features primarily contribute to failures. Failure recovery concerns moving the system back into its operating regime to continue deployment. This is challenging in that the policy is definitionally out-of-distribution, so alternative methods of operation must be used.

We propose an agentic approach to failure detection and recovery for humanoid foundation models. Distinct failure modes are addressed through unique recovery tools including locomotion, perceptive scanning, and manipulation. An orchestrator then identifies failure modes based on current observations during deployment and employs these tools.

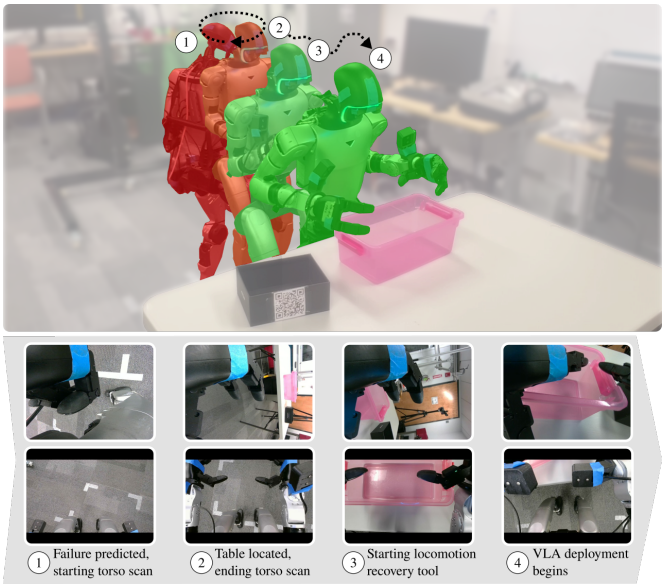


Fig. 1. Robot pose-based failure and locomotion recovery. In this demonstration, the humanoid is initialized several steps away from its intended workspace and turned to face a wall. Pre-task failure detection recognizes this poor initialization and performs a scan of the environment to locate the table. Then, a locomotion tool moves the robot to the table where it can then begin VLA inference.

Policy evaluation is performed in simulation through systematic perturbations to the robot and scene to build a representation for the operating regimes of the foundation model at hand, in this work $\pi_{0.5}$ [1] . Based on this evaluation, we learn a detector for identifying initial condition-based failures from visual features that mitigate the sim2real gap and allow this detector to be deployed zero-shot on hardware after training solely on simulation evaluation data.

This failure detection and recovery framework (see Figure 1) recovers from incorrect robot initialization, incorrect scene initialization, task failures, and policy stalls. Trajectories from the recovery framework are also stored in an offline buffer to iteratively improve the individual recovery tools according to the results of their interventions.

The main contributions of this work are summarized as:

1) Synthetic robustness analysis of fine-tuned policies via perturbations to initial robot and scene conditions to learn a failure detector via intermediate visual representations for zero-shot sim2real deployment
2) Agentic failure detection and recovery framework that

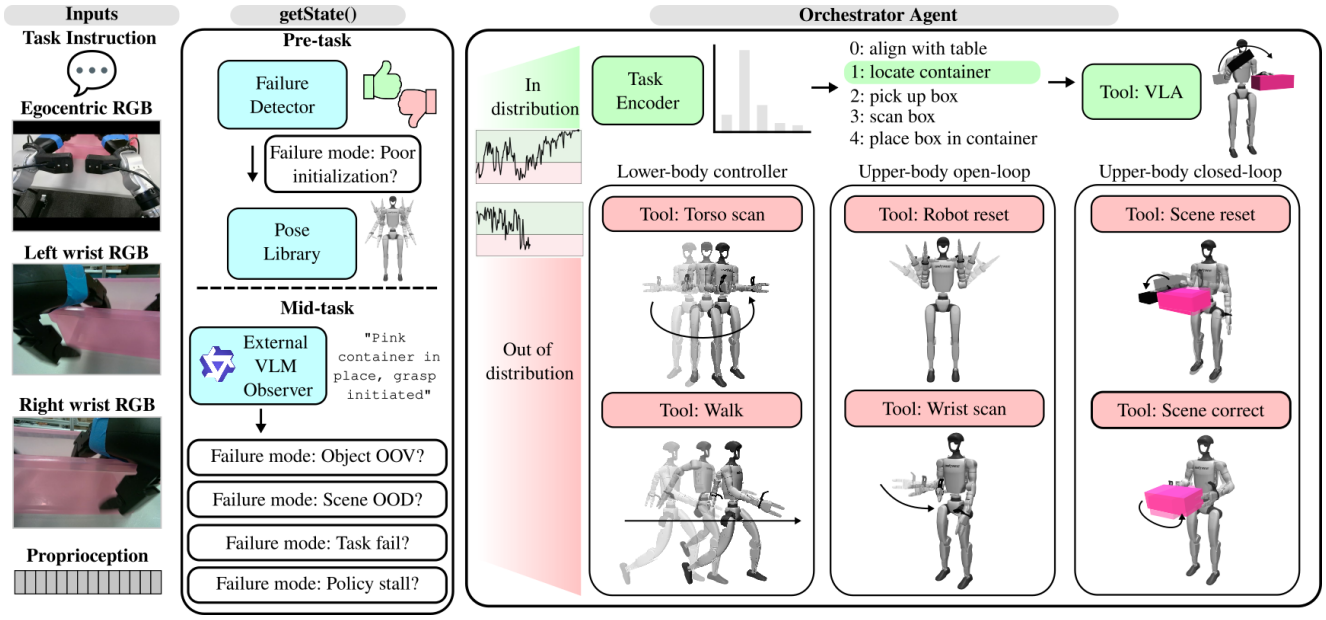


Fig. 2. Information flow for the proposed failure detection and recovery framework. At a given timestep, the system state is queried. Before a task is begun, a learned failure detector dictates whether or not the initial robot pose is appropriate. If the VLA has begun inference, an external VLM observer monitors progress to determine if various failure modes have occurred.

employs a generalizable set of tools for locomotion and manipulation failures
3) Online improvement of grasp tools through learned residuals to align execution and recovery behavior
4) Ablative analysis of task encoder and failure detector modules across various architectures and data recipes
5) Deployment of this framework on the Unitree G1 humanoid robot

Figure 2 provides a visualization of this workflow.

II. Related Work

A. Humanoid Foundation Models

A large majority of robotic foundation models are designed for single-arm or bi-manual manipulation systems and tasks [1]–[3]. These foundation models are composed of (1) a VLM as a backbone which is typically pretrained on large-scale vision and language inputs and (2) an action expert which receives the intermediate latent features learned by the VLM to produce joint-level actions through imitation learning techniques. The action space spans the dimensions of the joint configuration for the arms along with the hands which can either be continuous joint positions or discrete open/closed flags. These models are deployed on humanoids through decoupled control of the upper and lower body.

An additional set of models are designed for whole-body loco-manipulation [5], [6]. For whole-body planning, auto-regressive action experts [5] decode latent tokens into joint targets, or diffusion transformers and flow-matching policies [6] denoise random samples into actionable trajectories.

This proposed work aims to mitigate the brittle performance of these imitation learning policies through a set of tools ranging from exploration to corrective control.

B. Failure Detection for Foundation Models

Foundation model failures can be analyzed through two main approaches, the first being out-of-distribution (OOD) detection. This task can be viewed as anomaly detection [7] either in the input and output space of the model [8]–[10] or in the internal parameter space of the model itself [11].

The second approach is failure detection, which can be viewed as a classification problem typically performed through supervised approaches [10], [12], [13]. Supervised approaches require labels for success and failure. Practically, this means running the system in both settings which can be time-consuming and dangerous. Therefore, finding ways to collect these labels in a safe and efficient way is important.

Our framework exploits high fidelity simulations to collect these class labels and context-rich latent features to allow them to transfer meaningfully to the real world.

C. Failure Recovery for Foundation Models

Once a failure has been detected at test-time, the challenge shifts to intervening to recover the system back into distribution to continue with deployment.

The simplest method of intervention for a failed trial is manual takeover [3]. In this scenario, a human operator takes control and moves the system to either reset it or complete the task. Then, these recovering trajectories can be used to refine the original model via methods such as reinforcement learning [14]. The clear drawback of human intervention is that the deployment pipeline then requires a human-in-the-loop to both monitor and recover the system which limits the overall autonomy of the framework.

Recently, ensemble and agentic frameworks have risen in popularity [15]–[17] due to their ability to reason through failures and autonomously dispatch different tools and behaviors according to the current scene and failure mode.

This work leverages these agentic approaches to cater unique behaviors to the separate locomotion and manipulation facets of humanoid foundation model deployment.

III. Preliminaries

This section details the base foundation model (Section III-A) used within the proposed framework, data collection (Section III-B), and policy fine-tuning (Section III-C).

A. $\pi_{0.5}$ Foundation Model

This proposed framework employs the $\pi_{0.5}$ VLA [1]. As inputs, the model accepts at timestep t an egocentric RGB image $\mathbf{I}_t^{\text{ego}} \in \mathbb{R}^{480\times640\times3}$ along with two additional images from the left and right wrist cameras $\mathbf{I}_t^{\text{left}}, \mathbf{I}_t^{\text{right}} \in \mathbb{R}^{480\times640\times3}$. In addition, upper-body robot proprioception $\mathbf{q}_t^{\text{upper}} \in \mathbb{R}^{16} := [\mathbf{q}_t^{\text{left}}, g_t^{\text{left}}, \mathbf{q}_t^{\text{right}}, g_t^{\text{right}}]$ is passed in where $\mathbf{q}_t^{\text{left}}, \mathbf{q}_t^{\text{right}} \in \mathbb{R}^7$ are the joint spaces for the left and right arms and $g_t^{\text{left}}, g_t^{\text{right}} \in \mathbb{R}$ are the binary open/closed signals for the left and right hands. Finally, the user provides a text prompt ℓ_t for the given task.

B. Data collection techniques and dataset recipes

Data was collected for table-top bi-manual manipulation tasks using the teleoperation framework visualized in Figure 3. The human operator sent commands through the PICO 4U Enterprise headset and handheld controllers via the `xr_teleoperate` [18] workflow.

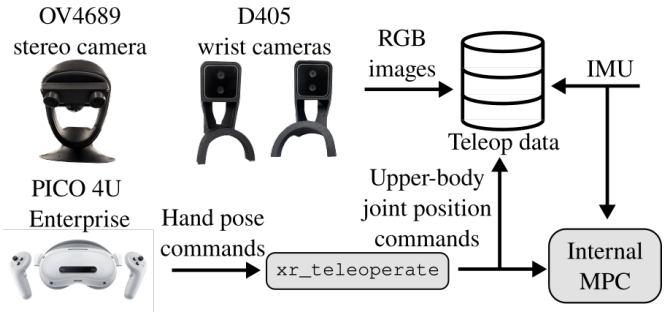


Fig. 3. Teleoperation workflow during data collection.

C. Model fine-tuning

The action expert is fine-tuned on an NVIDIA A100 GPU for 10,000 steps using a global batch size of 32 and a cosine decay learning rate scheduler with an initial rate of $2.5\text{e}-5$. This training stage took four hours to complete.

IV. Technical Core

In this section, we detail the failure detection and recovery framework proposed in this work (see Figure 2). Section IV-A details how failure detection is learned via systematic simulation perturbations and transferred over to hardware through latent featurization. Then, Section IV-B introduces a learned task encoder used for online subtask decomposition. Section IV-C discusses the series of tools that an orchestrator agent can utilize to recover a policy once failure is predicted, and Section IV-D showcases how these tools are improved via previously collected datasets.

A. VLA Failure Detection

1) Fine-tuned policy evaluation via simulation perturbations: First, the proposed framework performs policy evaluation in simulation to understand where the fine-tuned model can complete the given task. Evaluation is performed in simulation to avoid systematic benchmarking on hardware, a less reliable and more dangerous procedure. We extend the Unitree Isaac Sim environment [19] to support systematic perturbations to the initial pose of the humanoid torso $(\mathbf{p}_t^{\text{torso}}, \boldsymbol{\theta}_t^{\text{torso}})$ as well as the initial position of the end-effectors $(\mathbf{p}_t^{\text{left}}, \mathbf{p}_t^{\text{right}})$. Torso poses that collide with the scene are rejected, and hand positions that are kinematically infeasible or self-colliding are also rejected.

This evaluation illuminates the initial robot poses from which the policy is capable of completing its task. While these varied initial conditions can also be used to collect training data for fine-tuning, collecting curated demonstrations across a wide range of initial conditions is time-consuming and could harm performance if the policy forgets previously learned behaviors [20]. Evaluation results are discussed in more detail in Section V-B.

2) Learned Failure Detection: Policy evaluation yields success and failure labels, so now failure prediction can be learned. Given that policy evaluation data is recorded in simulation, it is necessary to encode input information into representations that enable zero-shot sim2real transfer.

First, egocentric RGB images are encoded into latent features via frozen visual and text encoders, namely the CLIP ViT-B/16 model [21]. To capture overall semantic information, global CLIP scores $\mathbf{V}_{\text{glob,t}}$ are calculated for the current timestep t along with 5- and 15-frame temporal differences. To capture spatial information, a 14×14 patch/text similarity map is converted into a set of 26 spatial statistics $\mathbf{V}_{\text{sp,t}}$ that capture semantic strength, position, mass, and area. These spatial statistics are well-preserved during sim2real transfer compared to the raw sensor information. The series of ablative experiments that led to this architecture are detailed in Section V-C.

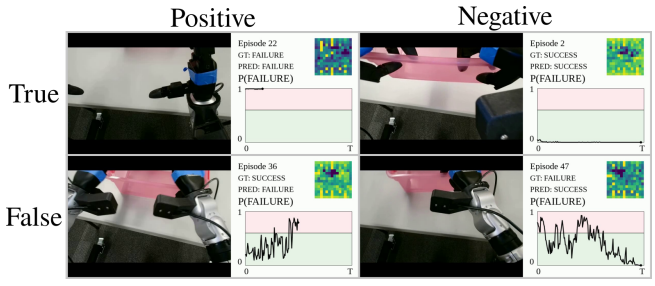


Fig. 4. Example predictions for the learned failure detector. For more consistent predictions, the detector must output a consistent label for $N_{\text{persist}} = 30$ frames before the label is accepted.

Second, relative proprioception $\mathbf{q}_t^{\text{rel}} = \mathbf{q}_t - \mathbf{q}_0$ and joint target residuals $\mathbf{q}_t^{\text{res}} = \mathbf{q}_t^{\text{tgt}} - \mathbf{q}_t$ are used as input. Relative and residual metrics survive the sim2real gap due to their enhanced robustness to varied initial configurations.

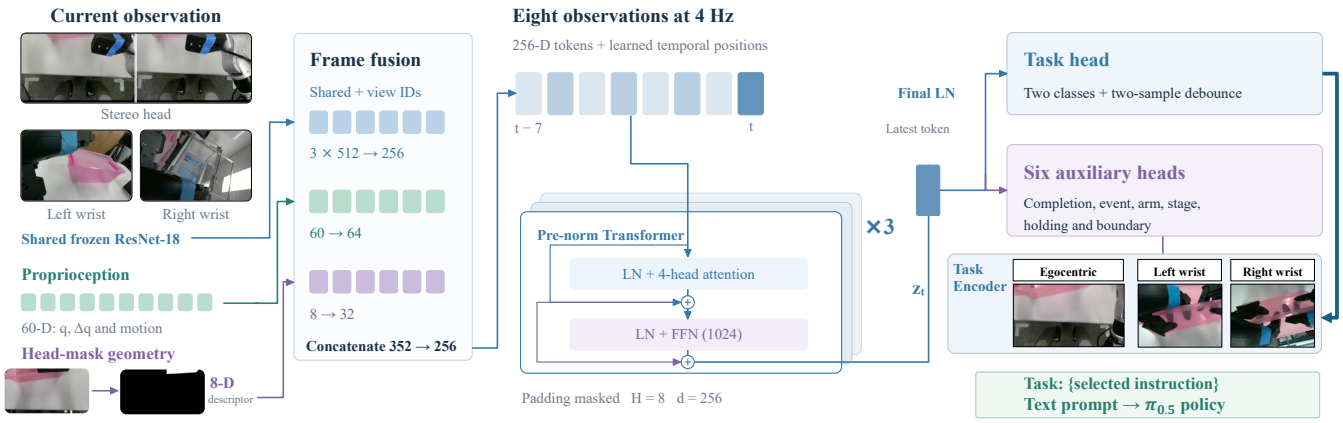


Fig. 5. Task encoder for state-dependent subtask selection. Visual, proprioceptive, and mask-geometry features are fused into frame tokens and processed by three Transformer blocks. The latest-token representation drives task selection and auxiliary semantic predictions. The selected instruction, recent events, and current semantic state form the language input to $\pi_{0.5}$. Note, FFN is Feed Forward Network, and LN is Linear Norm.

This set of features $\boldsymbol{\xi}_t^{\text{fd}} = \begin{bmatrix}\mathbf{V}_{\text{sp,t}} & \mathbf{q}_t^{\text{rel}} & \mathbf{q}_t^{\text{res}}\end{bmatrix} \in \mathbb{R}^{61}$ is then passed into a logistic regression classifier. Example predictions are visualized in Figure 4. These predictions are performed on a set of held out evaluation episodes from simulation. Hardware demonstrations that were collected for fine-tuning were used for additional evaluation and visualization. Crucially, this hardware data was not trained on.

3) Pose Search: Once a failure has been predicted, the detector also queries a library of previously observed successful robot poses for a corrective target. The recovery module is non-learned: it performs a nearest-neighbor search over recorded successful simulation rollouts.

B. Task Encoder for Subtask Decomposition

Our $\pi_{0.5}$ policy supports multiple subtasks, such as dragging, picking up, and scanning objects. An online task encoder selects and switches the policy's text prompt to execute subtask sequences, such as bringing a pink container before picking it up (Figure 5).

We supervise the encoder using demonstration annotations generated offline by `Qwen/Qwen3-VL-8B-Instruct`. For each temporal window, the annotator records events, outcomes, active arms, objects, targets, motion directions, failure reasons, and confidence, while tracking per-hand holding states, gripper states, and manipulation stages. Deterministic guardrails correct inconsistent state assignments, and annotation confidence weights semantic supervision. At deployment, the trained encoder selects prompts without invoking the vision-language annotator, enabling fast task switching.

1) Multimodal Temporal Encoding: At $4\,\text{Hz}$, the encoder processes eight observations spanning $1.75\,\text{s}$. Each observation combines three 512-D camera features from a shared, frozen, ImageNet-pretrained ResNet-18; 60-D proprioception comprising positions, causal differences, gripper norms, and arm-motion magnitudes; and an 8-D head-camera mask descriptor encoding validity, centroid, area, orientation, and orientation confidence. Proprioception is normalized using training-split statistics.

Shared camera projections and learned view embeddings precede fusion into 256-D frame tokens, followed by LayerNorm (LN), GELU, and dropout (0.15). Learned temporal embeddings and three pre-normalized Transformer blocks encode the sequence, using four attention heads and $1{,}024$-D feed-forward layers. Startup padding is masked; no triangular attention mask is needed because the window contains only current and past observations. Final LayerNorm and latest-token readout yield $\mathbf{z}_t \in \mathbb{R}^{256}$.

2) Training Objective: Seven linear heads predict subtask identity, completion, event, active arm, manipulation stage, per-hand holding state, and semantic boundary. We optimize

$$\mathcal{L}_{\text{enc}} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{complete}} + 0.5\mathcal{L}_{\text{sem}}, \tag{1}$$

$$\mathcal{L}_{\text{sem}} = \mathcal{L}_{\text{event}} + 0.45\mathcal{L}_{\text{arm}} + 0.65\mathcal{L}_{\text{stage}} + 0.45\mathcal{L}_{\text{holding}} + 0.20\mathcal{L}_{\text{boundary}}. \tag{2}$$

Categorical heads use cross-entropy; completion, holding, and boundary heads use binary cross-entropy. Losses incorporate training-split rare-class weights and confidence-weighted semantic supervision.

3) Architecture Selection: Across three seeds, the Transformer achieves the highest mean validation composite score ($84.74 \pm 0.38\%$), narrowly ahead of the Flash-WAM-style runner-up ($84.66 \pm 0.55\%$). Although this difference is small relative to seed variability, the Transformer uses 26.2% fewer trainable parameters (2.56M versus 3.47M), with similar cached-feature latency (0.585 versus $0.593\,\text{ms}$). It also outperforms mean pooling ($83.04 \pm 0.22\%$) by 1.70 percentage points, at greater size and latency (0.45M parameters and $0.251\,\text{ms}$ for mean pooling). We therefore select the Transformer for its validation performance and efficiency relative to its closest competitor. Scores are mean $\pm$ SD; latencies exclude image encoding.

C. Failure Recovery via Orchestrator

During task execution, we use `Qwen/Qwen-Coder 30B-A3B` [22] as an LLM orchestrator. Given a high-level user instruction (e.g., "pick up the pink container"), the orchestrator reasons over the task and invokes a set of tools

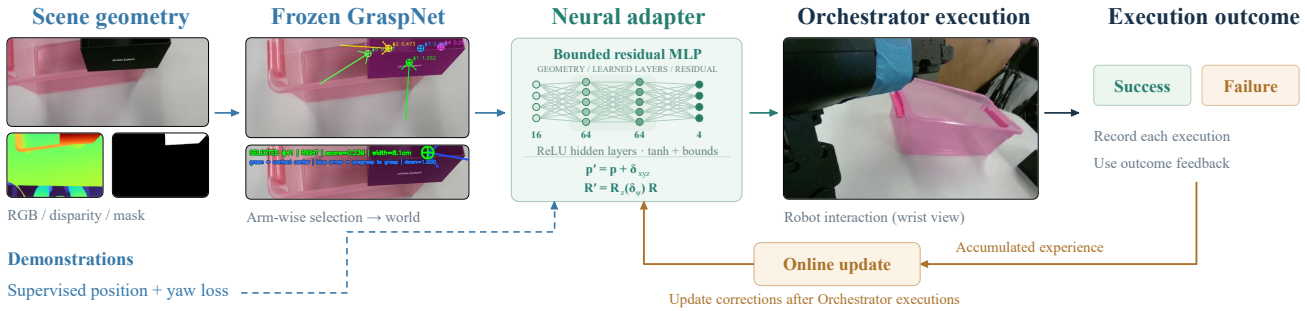


Fig. 6. Demonstration-trained neural adapter for the Orchestrator. Frozen GraspNet proposals supply geometric features to a 16–64–64–4 MLP. A scaled $\tanh$ bounds the predicted translation and world-yaw residuals, which correct proposal poses while preserving their scores and source indices. The lower strip reports held out mean position and yaw errors for frozen proposals and the two correction bounds. The execution-feedback loop refines the bounded adapter online, achieving a measured mean position error of $2.84\,\mathrm{cm}$, approximately 52.0% below its offline mean error.

to execute. In parallel, `Qwen/Qwen3-VL-8B-Instruct` (same architecture as the offline annotator in Section IV-B) continuously analyzes the robot's visual observations to describe the current scene and assess whether the observed state indicates progress toward the task goal. This information is converted to text and provided to the LLM as additional context for decision making in addressing the failure modes discussed in Section IV-C.1.

1) Failure modes: We outline five distinct failure modes from which this proposed framework can recover. These failure modes include (1) Poor torso pose initialization - the robot pose relative to the table is OOD, meaning the robot is either too far from the table or is not oriented with the table, (2) Poor arm pose initialization - the initial arm poses may be OOD or may block the view of the egocentric camera, (3) Target scene object out of view (OOV) - the current egocentric camera view does not include all elements of the scene relevant to task completion, (4) Target scene object out of distribution (OOD) - scene elements can be perceived by the current camera view, but their poses do not fall within the distribution of training data, (5) Task failure - the policy fails an intermediate step of the overall desired task, and (6) Policy stall - the policy ceases to move the robot any further through the task, leading the robot to idle or move but not attempt the task any longer. Note, these failure modes can be detected by the orchestrator, as it can call the VLA failure detector (Sec. IV-A) and from the VLM scene descriptions.

2) Recovering tools: To address these failure modes, six different recovery tools or behaviors are constructed which the orchestrator agent has access to and can instantiate according to its current observations. These tools include (1) Torso Scan - point wrist cameras outwards and rotate in place to scan surroundings, (2) Locomotion - apply (x, y) and yaw commands to the robot torso that are tracked by the lower-body controller, (3) Pose Reset - move the upper body into a default pose, (4) Wrist Scan - point a single wrist camera outwards and sweep to scan surroundings, (5) Scene Correction - re-orient an object within the scene to align it with training data, and (6) Scene Reset - pick and place objects within the scene to return them to an initial pose that aligns with training data. The first two tools employ the onboard Model Predictive Controller (MPC) to track commands, and the last two tools employ GraspNet [23] to grasp planning.

D. Recovering Tool Improvement

As the orchestrator relies on the performance of its given tools, such as GraspNet, we can utilize the data derived from the collected demonstrations and further refine the tool during actual online deployment.

Specifically, to improve the grasp proposals available to the orchestrator, we train a lightweight neural residual adapter from robot demonstrations while keeping GraspNet frozen (Figure 6). The adapter predicts bounded Cartesian and world-yaw corrections for each proposal.

1) Grasp proposal generation: We reconstruct metric stereo depth, run GraspNet on the segmented object cloud, and transform candidate poses into the robot world frame using the recorded robot state and camera calibration associated with the stored images.

2) Demonstration supervision: We derive grasp targets from recorded robot proprioception. Gripper-closing commands identify candidate grasp intervals, while forward kinematics provides the corresponding palm pose. Accounting for the displacement from the controlled palm site to its outward surface yields the target position; the palm orientation provides the target yaw. These are kinematic targets rather than direct measurements of object contact. Matching proposals to these targets provides supervision for learning position and yaw corrections.

Extraction produces 219 valid arm records, of which 66 records from 50 trajectories yield 150 matched GraspNet candidate–demonstration pairs. We use a fixed trajectory-level split with 40 training trajectories (114 pairs) and five validation trajectories (36 pairs), keeping all pairs from each demonstration in the same partition to prevent data leakage.

3) Bounded neural residuals: For each proposal, let $\mathbf{p} \in \mathbb{R}^3$, $\mathbf{R} \in SO(3)$, and ψ denote its position, orientation, and yaw, respectively. The adapter processes 16 geometric features normalized using statistics stored in the checkpoint. With $\boldsymbol{\eta}^{\mathrm{grasp}} \in \mathbb{R}^{16}$ denoting the normalized feature vector, a 5,508-parameter multilayer perceptron f_ω uses layers $16 \to 64 \to 64 \to 4$ with ReLU activations after the two hidden layers. Its output is bounded as

$$\boldsymbol{\delta} = \mathbf{b} \odot \tanh(f_\omega(\boldsymbol{\eta}^{\mathrm{grasp}})), \qquad \boldsymbol{\delta} = [\delta_x, \delta_y, \delta_z, \delta_\psi]^\mathsf{T}, \quad (3)$$

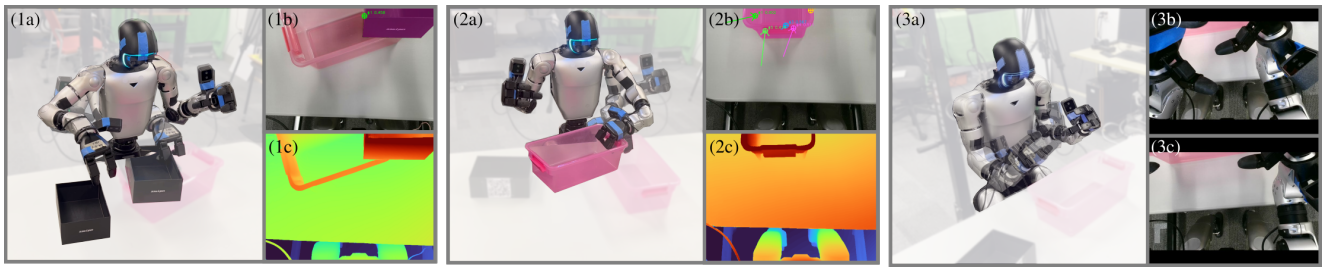


Fig. 7. Hardware overlay of different recovery tools. Overlay (1a) shows a scene reset where the black box is picked from the pink container and placed back onto the table for a new task attempt. (1b) shows the selected grasp from GraspNet while (1c) shows the disparity image uses within GraspNet. Overlay (2a) shows a scene correction where the pink container is re-oriented to align with its pose from VLA training demonstrations. Overlay (3a) shows a pose reset to move the arms from blocking the egocentric view (3b) to a pose that allows for clear observation (3c).

where $\odot$ denotes elementwise multiplication. The intended bounded model uses $\mathbf{b} = [0.05, 0.05, 0.05, \pi/6]^\mathsf{T}$, with translation in metres and yaw in radians. A relaxed variant uses $\mathbf{b} = [0.15, 0.15, 0.15, \pi/2]^\mathsf{T}$. Writing $\boldsymbol{\delta}_{xyz} = [\delta_x, \delta_y, \delta_z]^\mathsf{T}$, the corrected proposal is

$$\mathbf{p}' = \mathbf{p} + \boldsymbol{\delta}_{xyz}, \tag{4}$$

$$\mathbf{R}' = \mathbf{R}_z(\delta_\psi)\mathbf{R}, \qquad \psi' = \mathrm{wrap}(\psi + \delta_\psi), \tag{5}$$

where $\mathbf{R}_z$ rotates about the world z-axis and wrap maps angles to $[-\pi, \pi]$.

4) Training: For a matched demonstration target with position $\mathbf{p}_i^{\mathrm{demo}}$ and yaw ψ_i^{demo}, the demonstration loss is

$$\mathcal{L}_{\mathrm{demo}} = \frac{1}{N_{\mathrm{demo}}} \sum_i \left[\|\mathbf{p}_i' - \mathbf{p}_i^{\mathrm{demo}}\|_2 + \kappa \left| \mathrm{wrap}(\psi_i' - \psi_i^{\mathrm{demo}}) \right| \right], \tag{6}$$

with $\kappa = 0.05\,\mathrm{m/rad}$. The total objective is $\mathcal{L}_{\mathrm{grasp}} = \mathcal{L}_{\mathrm{demo}} + 0.05\mathcal{L}_{\mathrm{res}}$, where $\mathcal{L}_{\mathrm{res}}$ penalizes squared residual corrections, using the same yaw-to-distance scale κ. We train for 250 epochs using AdamW with learning rate 10^{-3}, weight decay 10^{-5}, batch size 64, and seed 17.

5) Held-out evaluation: On the 30 held-out trajectories, the bounded adapter reduces mean position error from 12.35 to $5.92\,\mathrm{cm}$ (52.1%) and mean yaw error from 38.34° to 16.65°. The relaxed variant achieves $1.74\,\mathrm{cm}$ and 7.37°, corresponding to an 85.9% reduction in mean position error. Position RMSE is 12.40, 6.15, and $2.04\,\mathrm{cm}$ for frozen, bounded, and relaxed proposals, respectively. The 95% confidence intervals for mean position error are $[11.91, 13.02]$, $[5.18, 6.74]$, and $[1.37, 2.60]\,\mathrm{cm}$; these use 2,000 trajectory-level bootstrap resamples. These results evaluate matched-pair pose accuracy on a small held-out set, not physical grasp success. A separate 1,000-epoch relaxed checkpoint trained on all 150 pairs achieves $1.41\,\mathrm{mm}$ mean position error and 0.337° mean yaw error on those same pairs.

6) Online refinement: The inference function batches all GraspNet proposals, applies the predicted position and world-yaw corrections, and preserves each proposal's original score and source index. Following offline training, we deploy the bounded neural adapter within the orchestrator and refine it using accumulated execution experience. After each execution, we record the applied correction and observed success or failure, using this feedback to guide subsequent adapter updates while retaining the prescribed output bounds. Online refinement achieves a measured mean position error of $2.84\,\mathrm{cm}$, approximately 52.0% lower than the bounded offline adapter's $5.92\,\mathrm{cm}$. This online measurement is reported separately from the trajectory-held-out offline evaluation.

V. Experimental Results

A. Experimental setup

In this section, we detail the experimental setup in which this proposed framework is applied.

1) Robot: The robot state is

$$\mathbf{q}_t^{\mathrm{robot}} = \begin{bmatrix} \mathbf{q}_t^{\mathrm{lower}} & \boldsymbol{\theta}_t^{\mathrm{torso}} & \mathbf{q}_t^{\mathrm{arms}} & \mathbf{q}_t^{\mathrm{hands}} \end{bmatrix} \tag{7}$$

where $\mathbf{q}_t^{\mathrm{lower}} \in \mathbb{R}^{14}$ is the lower-body state of the left and right legs, $\boldsymbol{\theta}_t^{\mathrm{torso}} \in \mathbb{R}^3$ is the torso orientation, $\mathbf{q}_t^{\mathrm{arms}} \in \mathbb{R}^{14}$ is the upper-body state of the left and right arms, and $\mathbf{q}_t^{\mathrm{hands}} \in \mathbb{R}^{14}$ is the state of the left and right hands.

An OV4689 stereo camera provides both RGB and depth images and RealSense D405 RGB-D cameras provide left and right wrist views. The authors found that manual calibration of these cameras led to improved performance.

2) Tasks: We deploy this framework on three distinct tasks which are discussed in this section.

a) Task 1: Bring container into view: To explicitly instruct the foundation model to bring target items into view, we defined the first task as locating target scene objects, in this case a pink container, and manipulating them to bring them into view for the egocentric camera. Given that the neck of the robot is not jointed, the robot must use the wrist cameras to search the scene and locate target objects. Once found, the robot can then grasp the object and drag it into view. On hardware, 51 episodes are collected which totals 19,662 frames or 10.92 minutes of data, with each episode taking an average of 12.9 seconds. For simulation evaluation, task completion is defined by the container center lying within the egocentric camera image.

b) Task 2: Pick up container: The robot must grasp a pink container and raise it above the level of the table. On hardware, 51 episodes are collected which totals 11,989 frames or 6.66 minutes of data, with each episode taking an average of 7.8 seconds. For simulation evaluation, task completion is defined by the container center having 12 cm of clearance from the table surface.

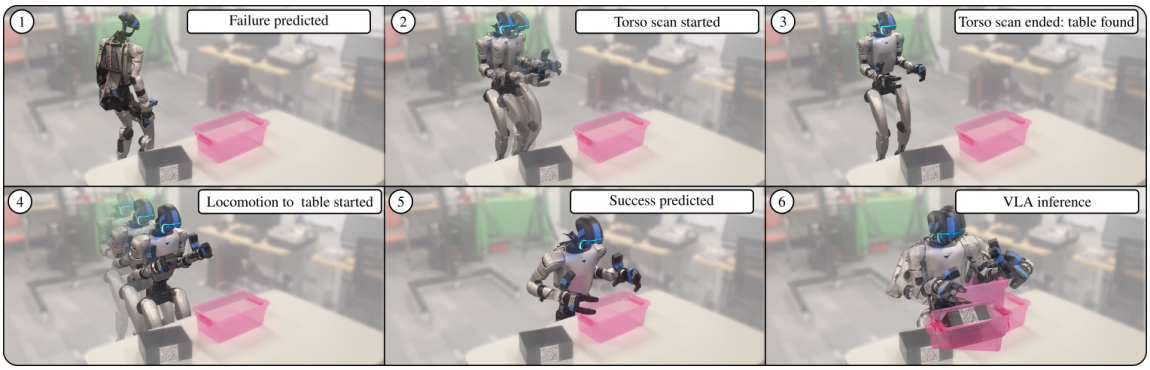


Fig. 8. Composition of stages for failure detection, recovery, and inference.

c) Task 3: QR code scanning: Lastly, the robot must again use its wrist camera to locate a target item, in this case a black box with a QR code. This black box is deliberately initialized to be partially out of view. The robot must first locate this box and grasp it with the right hand. Then, it must bring the box over to the left wrist camera to scan the QR code. Once the code is scanned, the black box is placed into a pink container. On hardware, 141 episodes are collected which totals 66,789 frames or 37.11 minutes of data, with each episode taking an average of 15.8 seconds. For simulation evaluation, task completion is defined two-fold: successful scan of the QR code from the left wrist camera (determined through QR scanner libraries), and placement of the box into the container.

B. Policy Evaluation

This section details the systematic perturbations applied to the fine-tuned policies. For brevity, only evaluations of the *Pick up container* task are shown here.

1) Yaw evaluation: Policy results under yaw perturbations are visualized in Figure 9. Yaws are binned between $\pm\frac{\pi}{2}$ with a step of $\frac{\pi}{36}$. The policy is tested at each yaw value over 25 trials, making for a total of 925 episodes.

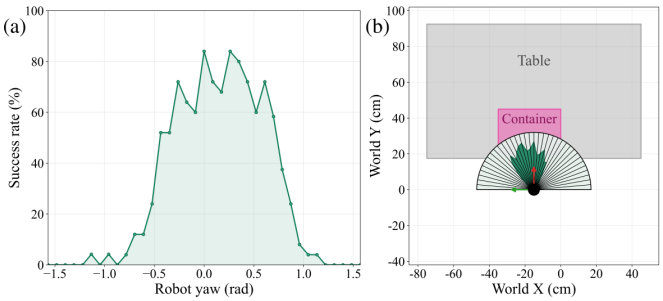


Fig. 9. Performance of fine-tuned $\pi_{0.5}$ policies under systematic perturbations in simulation. (a) Policy success rate (%) over robot yaw (rad). (b) Task space visualization of policy performance over robot yaw.

The policy experiences a drop-off in performance beyond deviations of $\pm\frac{\pi}{4}$. There is also a bias in successful trials towards positive yaws. Of the 925 episodes, 271 were successful and 160 of those were at positive yaws.

2) (x, y) evaluation: Policy results under (x, y) perturbations are visualized in Figure 10. The x positions are binned between 65 cm and 35 cm with a step of 10 cm. This range roughly spans the width of the table surface. The y positions are binned between -10 cm and 10 cm with a step of 5 cm. The operating regime is narrow in the (x, y) plane, with

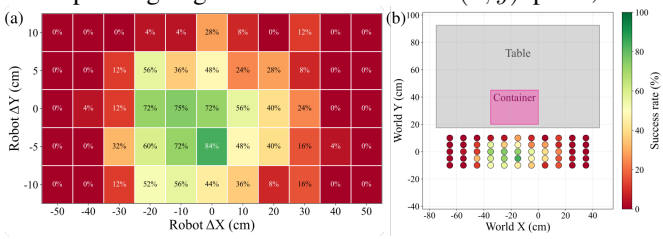


Fig. 10. Performance of fine-tuned $\pi_{0.5}$ policies under systematic perturbations in simulation.

the policy only showing consistent performance in roughly a 10 cm $\times$ 10 cm region. This suggests that locomotion recovery tools need a high degree of precision to move the system into a region where the policy can reliably succeed.

3) Hand position evaluation: Policy results under 3D hand position perturbations are visualized through 2D heatmaps in Figure 11. To minimize the scaling of perturbing both hand positions individually, symmetric displacements are applied to nominal hand positions calculated from training demonstrations. Hand positions are checked for feasibility based on reachability and self-collisions.

The $\pi_{0.5}$ policy demonstrated different levels of robustness across the three dimensions evaluated. Performance across Δx values were varied, with success rate degrading heavily as hands were moved further out into the scene. On the contrary, performance across Δy values was fairly consistent.

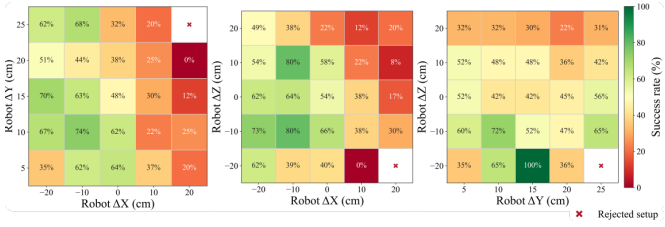


Fig. 11. Performance of fine-tuned $\pi_{0.5}$ policies under systematic perturbations in simulation.

C. Failure detector ablative analysis

First, we perform ablative analysis on three visual encoders to determine which encoders and visual features best preserve failure prediction signals on hardware. The DINOv2 [24], SigLIP [25], and CLIP [21] encoders are all evaluated.

Parameter sweeps are performed on each encoder to determine best performance in simulation (sim-best).

TABLE I

OPERATING POINTS EVALUATED ZERO-SHOT ON HARDWARE.

Representation	Simulation BA	Hardware Recall	Hardware FPR	Hardware BA
DINOv2 sim-best	87.8	100.0	49.0	75.5
SigLIP sim-best	**88.4**	45.1	0.0	72.5
CLIP sim-best	84.3	19.6	0.0	59.8
SigLIP text + $\Delta_{5,15}$	80.3	100.0	19.6	90.2
CLIP text + $\Delta_{5,15}$	82.6	100.0	**2.0**	**99.0**

All values are percentages. BA: balanced accuracy; FPR: success false-exit rate.

Incorporating $5-$ and $15-$ step deltas for features resulted in high accuracy during hardware evaluation for the CLIP model. Next, we isolate the CLIP encoder and perform ablation against various proprioceptive representations.

TABLE II

SCALAR AND SPATIAL CLIP ABLATIONS

Inputs	Simulation AUC	Simulation BA	Hardware AUC	Hardware Recall	Hardware FPR	Hardware BA
$\mathbf{V}_{\text{glob}}$	79.1	80.0	96.0	100.0	17.6	91.2
$\mathbf{V}_{\text{glob}} + \mathbf{q}^{\text{rel}}$	85.4	82.8	89.5	86.3	0.0	93.1
$\mathbf{V}_{\text{glob}} + \mathbf{q}^{\text{res}}$	84.9	84.7	94.5	100.0	98.0	51.0
$\mathbf{V}_{\text{glob}} + \mathbf{q}^{\text{rel}} + \mathbf{q}^{\text{res}}$	86.6	83.0	**99.5**	100.0	**0.0**	**100.0**
$\mathbf{V}_{\text{sp}}$	87.4	84.3	96.5	98.0	5.9	96.1
$\mathbf{V}_{\text{sp}} + \mathbf{q}^{\text{rel}}$	88.7	86.5	90.7	94.1	**0.0**	**97.1**
$\mathbf{V}_{\text{sp}} + \mathbf{q}^{\text{res}}$	88.6	87.7	97.2	100.0	25.5	87.3
$\mathbf{V}_{\text{sp}} + \mathbf{q}^{\text{rel}} + \mathbf{q}^{\text{res}}$	**88.9**	**90.3**	98.6	94.1	**0.0**	**97.1**

All values are percentages. $\mathbf{V}_{\text{sp}}$: spatial CLIP; AUC: area under curve.

Incorporating both relative proprioception information as well as residual joint targets improved performance when compared to a purely visual detector.

D. Composite tool demonstration

Figure 7 depicts a set of manipulation recovery tools that allow the system to reset and correct the current scene and additionally reset the robot pose to begin VLA inference. Under the initial conditions in each setting, the stand-alone policy failed without recovery tools. Figure 8 depicts the proposed framework recovering from poor robot initialization by searching for and moving to the desired workspace via locomotion recovery tools and beginning VLA inference. At this initial pose, the policy alone would clearly fail. Across all hardware trials, the proposed framework increased overall task success rate by 50% over the VLA alone and completed $4/5$ trials for *Bring container into view*, $5/5$ trials for *Pick up container*, and $3/5$ trials for *QR code scanning*.

VI. CONCLUSION

In this work, we propose a failure detection and recovery framework for humanoid foundation models. An orchestrator uses semantic reasoning to dispatch various tools according to the presently detected failure. Pre-task failures are addressed through initial robot and scene adjustments via locomotion and manipulation controllers, respectively. Mid-task failures are addressed through robot and scene resets. To improve the capabilities of recovery tools at run-time, we train a residual network and apply these learned position and orientation offsets for grasps during recovery to align policy and tool actions. In the future, we aim to further improve recovering tools via online reinforcement learning.

REFERENCES

[1] K. Black *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," in *Proceedings of The 9th Conference on Robot Learning* (J. Lim, S. Song, and H.-W. Park, eds.), vol. 305 of *Proceedings of Machine Learning Research*, pp. 17–40, PMLR, 27–30 Sep 2025.

[2] P. Intelligence *et al.*, "$\pi^*_{0.6}$: a vla that learns from experience," 2025.

[3] P. Intelligence *et al.*, "$\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities," 2026.

[4] E. Ackerman, "Large behavior models are helping atlas get to work," Oct 2025.

[5] J. Bjorck *et al.*, "Gr00t n1: An open foundation model for generalist humanoid robots," 2025.

[6] S. Wei *et al.*, "ψ_0: An open foundation model towards universal humanoid loco-manipulation," 2026.

[7] M. Salehi, H. Mirzaei, D. Hendrycks, Y. Li, M. H. Rohban, and M. Sabokrou, "A unified survey on anomaly, novelty, open-set, and out-of-distribution detection: Solutions and future challenges," 2022.

[8] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina, "Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies," 2025.

[9] A. Majumdar, M. Sharma, D. Kalashnikov, S. Singh, P. Sermanet, and V. Sindhwani, "Predictive red teaming: Breaking policies without breaking robots," 2025.

[10] H. Liu, Y. Zhang, V. Betala, E. Zhang, J. Liu, C. Ding, and Y. Zhu, "Multi-task interactive robot fleet learning with visual world models," in *Proceedings of The 8th Conference on Robot Learning*, Proceedings of Machine Learning Research, PMLR, 06–09 Nov 2025.

[11] R. Sinha, A. Elhafsi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone, "Real-time anomaly detection and reactive planning with large language models," 2024.

[12] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, "SAFE: Multitask failure detection for vision-language-action models," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[13] C. Gokmen, D. Ho, and M. Khansari, "Asking for help: Failure prediction in behavioral cloning through value approximation," 2023.

[14] B. Thananjeyan *et al.*, "Recovery rl: Safe reinforcement learning with learned recovery zones," 2021.

[15] H. Hu, C.-T. Huang, J. Liu, Y. Wang, K. Suzuki, M. Brand, and T. Koike-Akino, "Recovla: Vlm-guided reward compilation for failure recovery in vision-language-action policies," 2026.

[16] Y. Dai, J. Lee, N. Fazeli, and J. Chai, "Racer: Rich language-guided failure recovery policies for imitation learning," 2024.

[17] Z. Lin, J. Duan, H. Fang, D. Fox, R. Krishna, C. Tan, and B. Wen, "Failsafe: Reasoning and recovery from failures in vision-language-action models," 2026.

[18] Unitree Robotics, "XR-Teleoperate: An open-source teleoperation framework and data collection toolkit for embodied intelligence." `https://github.com/unitreerobotics/xr_teleoperate`, 2024. Accessed: 2026-02.

[19] Unitree Robotics, "unitree_sim_isaaclab." `https://github.com/unitreerobotics/unitree_sim_isaaclab`, 2026. GitHub repository, accessed September 7, 2026.

[20] G. D. Team, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[21] O. R. Team, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning*, PMLR, 18–24 Jul 2021.

[22] Q. team, "Qwen3 technical report," 2025.

[23] H.-S. Fang, C. Wang, M. Gou, and C. Lu, "Graspnet-1billion: A large-scale benchmark for general object grasping," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11441–11450, 2020.

[24] M. A. R. Team, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024. Featured Certification.

[25] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11975–11986, 2023.